CORPORACIÓN UNIFICADA NACIONAL DE EDUCACIÓN SUPERIOR – CUN

Aca – Programación Avanzada

SISTEMA DE GESTIÓN DE BIBLIOTECA

Documento técnico del proyecto

Estudiante: Juan Andres Jaramillo **Código estudiantil:** [54465] **Programa:** Ingeniería de Sistemas
Docente: VERONICA CASTRO MUNAR

Medellín, Colombia Noviembre de 2026

Aca – Programación Avanzada

Sistema de Gestión de Biblioteca

Aplicación de escritorio desarrollada en C#, Windows Forms y SQL Server

Repositorio del proyecto: https://github.com/jaramillocode/BibliotecaApp

Juan Andres Jaramillo · 54465 Medellín, noviembre de 2026

Tabla de contenido

1. Introducción
2. Objetivos
3. Planteamiento del problema
4. Análisis de requerimientos
5. Casos de uso
6. Diagrama de clases
7. Modelo entidad-relación
8. Diccionario de datos
9. Arquitectura del sistema
10. Explicación de los módulos desarrollados
11. Conexión con SQL Server
12. Capturas de pantalla del sistema
13. Pruebas de funcionamiento
14. Conclusiones
15. Recomendaciones
16. Referencias bibliográficas

> **Numeración de páginas:** el formato Markdown no numera páginas por sí mismo. La numeración aparece al exportar este documento a PDF o Word (ver `GUIA_ENTREGA.md`, paso 8).

1. Introducción

Las bibliotecas de las instituciones educativas manejan a diario información que cambia rápido: libros que entran y salen, usuarios que se registran y préstamos que deben devolverse en alguna fecha. Cuando ese control se lleva en cuadernos u hojas sueltas es fácil que se pierdan datos, que un libro figure como disponible sin estarlo o que no se sepa quién lo tiene.

Este proyecto presenta el **Sistema de Gestión de Biblioteca**, una aplicación de escritorio desarrollada con **C#, Windows Forms y SQL Server** (plataforma .NET 8). El sistema permite administrar libros, autores, editoriales y usuarios, registrar préstamos y devoluciones, y consultar reportes sobre el estado de la biblioteca.

El código se organizó en **cuatro proyectos** (entidades, datos, negocio y presentación) siguiendo una arquitectura por capas. Se aplicaron los principios de la Programación Orientada a Objetos, se usaron consultas parametrizadas para todas las operaciones sobre la base de datos y se versionó el trabajo con Git y GitHub.

Este documento explica el problema, los requerimientos, el diseño (casos de uso, clases, modelo de datos y arquitectura), cada módulo construido, la conexión con SQL Server, las pruebas realizadas y las conclusiones del trabajo.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación de escritorio para la gestión integral de una biblioteca utilizando C#, Windows Forms y SQL Server, que permita administrar libros, autores, editoriales y usuarios, gestionar préstamos y devoluciones, y generar reportes mediante operaciones conectadas a una base de datos relacional, aplicando los principios de la Programación Orientada a Objetos.

2.2 Objetivos específicos

- Aplicar los principios de Programación Orientada a Objetos (clases, herencia, encapsulamiento, polimorfismo).
- Diseñar una base de datos relacional con claves primarias, foráneas y restricciones de integridad.
- Implementar operaciones CRUD para libros, autores, editoriales y usuarios.
- Utilizar SQL Server como gestor de base de datos.

- Implementar una arquitectura por capas que separe presentación, lógica de negocio y acceso a datos.
- Validar la información ingresada por el usuario y manejar excepciones con mensajes claros.
- Utilizar Git y GitHub como sistema de control de versiones.
- Documentar técnicamente el proyecto.
- Implementar formularios que permitan registrar, consultar, actualizar y eliminar información.

3. Planteamiento del problema

Una institución educativa necesita una aplicación de escritorio que facilite la administración de su biblioteca. La gestión de libros, autores, editoriales y usuarios, así como el control de préstamos y devoluciones, requiere un sistema organizado que permita registrar, consultar, actualizar y eliminar información, hacer seguimiento al estado de los préstamos y generar reportes.

Los problemas que el sistema resuelve son:

Problema	Cómo lo resuelve el sistema
La información bibliográfica está dispersa	Todo se guarda en una sola base de datos SQL Server
No se sabe cuántos ejemplares hay disponibles	Cada libro lleva `Existencias` y `Disponibles`, que se actualizan solos con cada préstamo y devolución
Se pueden registrar datos repetidos o incorrectos	Restricciones únicas en la base de datos y validaciones en la capa de negocio
No se sabe quién tiene un libro	Cada préstamo relaciona un usuario con un libro, con fechas y estado
Es difícil consultar el estado general	Módulo de reportes con 10 consultas y exportación a CSV

4. Análisis de requerimientos

4.1 Requerimientos funcionales

Código	Requerimiento	Dónde se implementa
RF01	Gestión de libros: registrar, listar, editar, eliminar, buscar por código/ISBN, título o	`FormLibros`, `LibroNegocio`, `LibroDatos`

Código	Requerimiento	Dónde se implementa
	autor, relacionar con autor y editorial, consultar disponibilidad	
RF02	Gestión de autores (id, nombre, apellido y nacionalidad)	`FormAutores`, `AutorNegocio`, `AutorDatos`
RF03	Gestión de editoriales (id único, nombre, país, teléfono)	`FormEditoriales`, `EditorialNegocio`, `EditorialDatos`
RF04	Gestión de usuarios (id, nombre, apellido, documento, teléfono, correo)	`FormUsuarios`, `UsuarioNegocio`, `UsuarioDatos`
RF05	Gestión de préstamos: seleccionar usuario y libro, registrar fecha, consultar, editar, eliminar y ver el estado	`FormPrestamos`, `PrestamoNegocio`, `PrestamoDatos`
RF06	Gestión de devoluciones: actualiza fecha, estado a «Devuelto» y disponibilidad del libro	`FormPrestamos` (botón *Registrar devolución*), `PrestamoDatos.Devolver`
RF07	Consultas y reportes	`FormReportes`, `ReporteNegocio`, `ReporteDatos`
RF08	Navegación del sistema mediante un formulario principal	`FrmPrincipal`

4.2 Requerimientos no funcionales

Requerimiento	Cómo se cumple
Interfaz clara, organizada y fácil de usar	Menú lateral fijo; todos los módulos de gestión comparten la misma plantilla (`FormModulo`)
Validación de campos obligatorios	Clase `Validador` en la capa de negocio; los obligatorios se marcan con `*`
Prevención de duplicados	Restricciones `UNIQUE` (ISBN, documento, nombre de editorial, nombre+apellido de autor) e índice único filtrado para préstamos activos

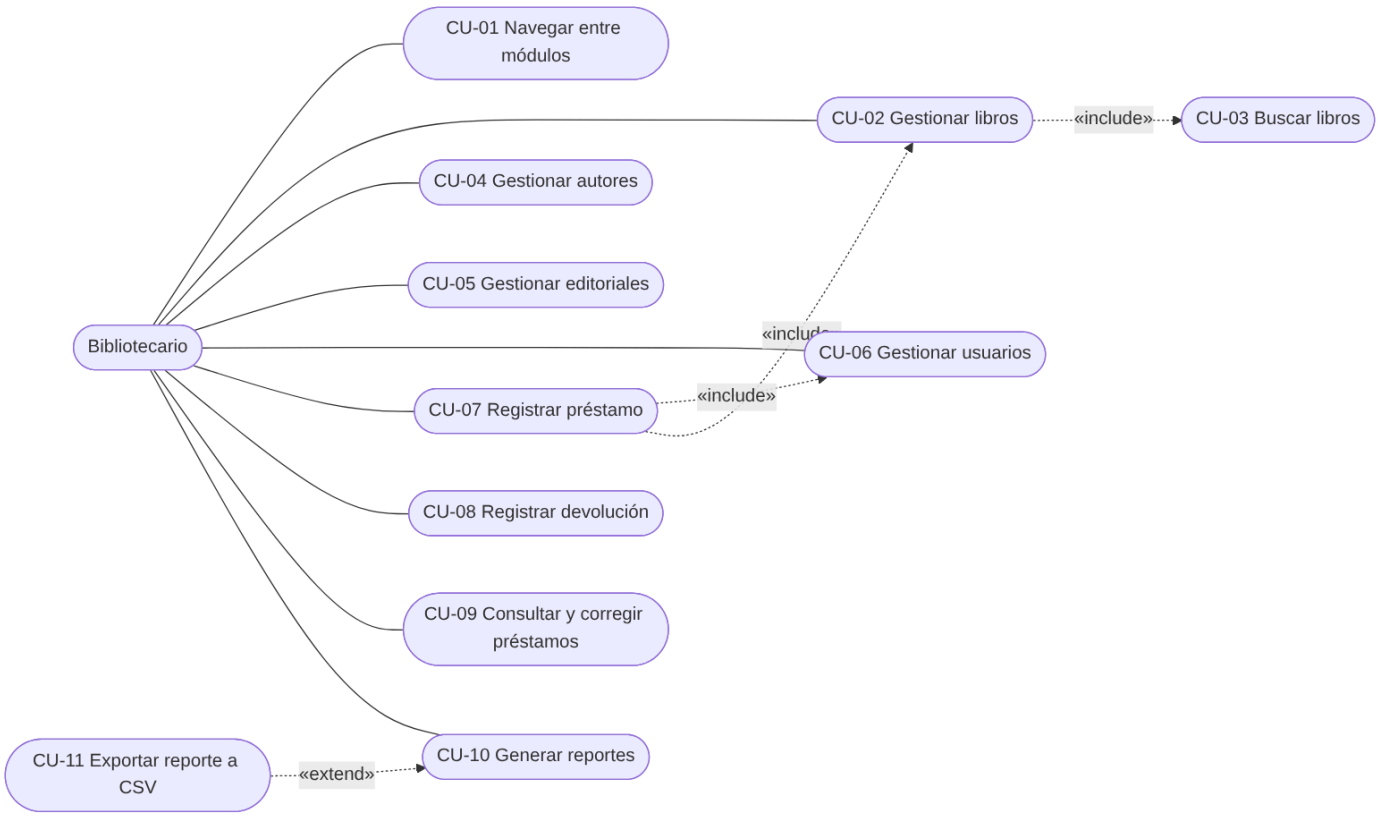
Requerimiento	Cómo se cumple
Mensajes claros de operaciones y errores	`Ui.Info`, `Ui.Advertencia`, `Ui.Error`; errores técnicos traducidos por `TraductorErrores`
Manejo adecuado de excepciones	`DatosException` (datos), `ReglaNegocioException` (negocio) y el método `Seguro()` en la interfaz
Código organizado y legible; nombres descriptivos	Cuatro proyectos, una clase por responsabilidad, nombres en español
Integridad de los datos	Claves foráneas, `CHECK` y transacciones en préstamos/devoluciones
Conexión adecuada con SQL Server	`ConexionBD` lee la cadena desde `App.config`; se prueba al iniciar
Consultas parametrizadas	Todas pasan por `Ejecutor` con `SqlParameter`
Separación de responsabilidades	Los formularios no contienen SQL; solo llaman a la capa de negocio

4.3 Reglas de negocio

1. Un libro no puede tener más ejemplares disponibles que existencias (`Disponibles <= Existencias`).
2. No se puede prestar un libro sin ejemplares disponibles.
3. Un usuario no puede tener el mismo libro prestado dos veces a la vez.
4. La fecha del préstamo y la de devolución no pueden ser posteriores a hoy, y la devolución no puede ser anterior al préstamo.
5. Un préstamo `Prestado` no tiene fecha de devolución; uno `Devuelto` siempre la tiene.
6. No se elimina un autor o editorial con libros, ni un libro o usuario con préstamos.
7. Al eliminar un préstamo activo, el ejemplar vuelve al inventario.
8. No se pueden dejar menos existencias que los ejemplares que están prestados.
9. Las fechas y el usuario/libro de un préstamo ya registrado no cambian el inventario: solo se corrigen fechas.

5. Casos de uso

Actor: Bibliotecario (persona que opera la aplicación).

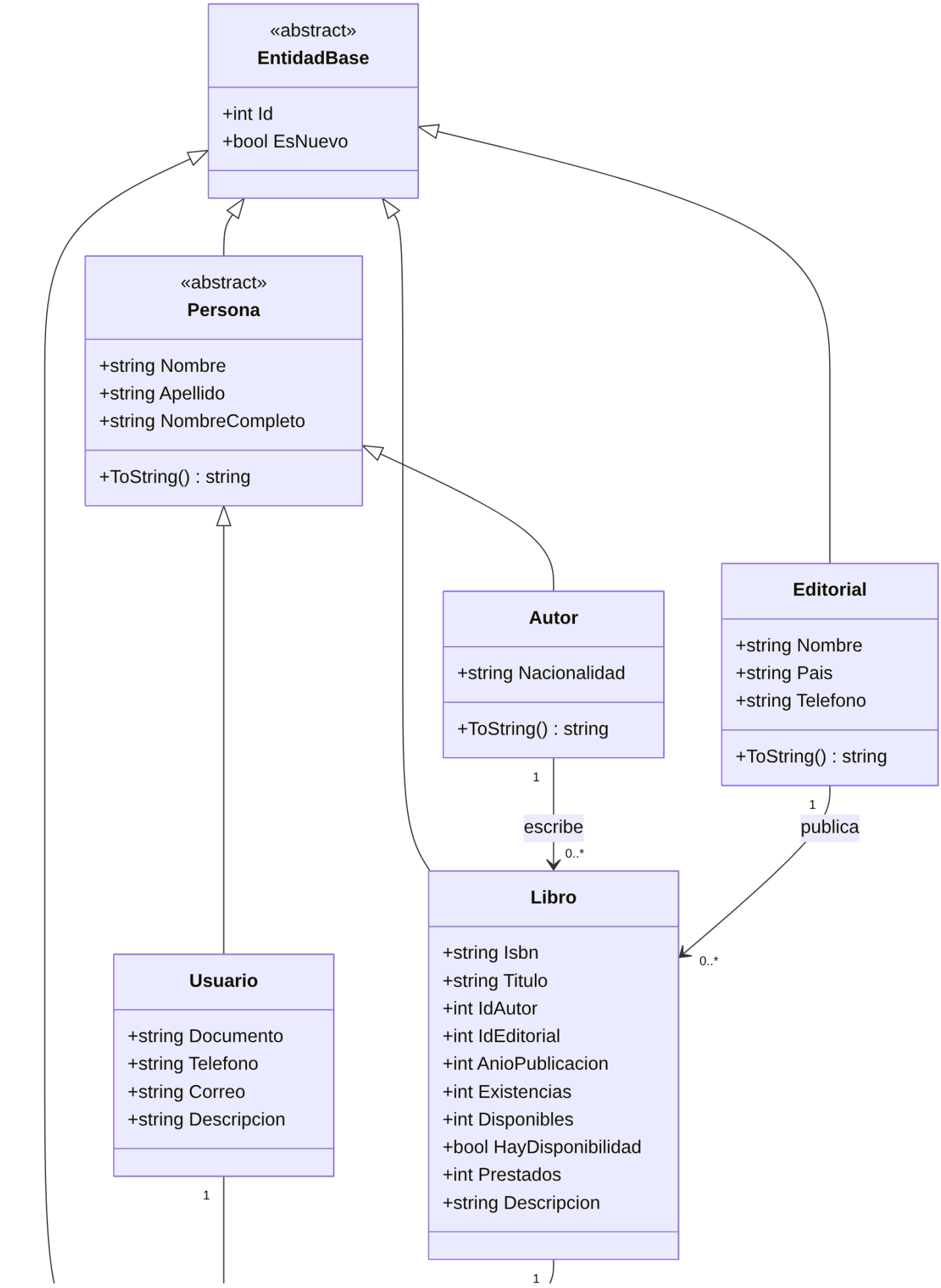


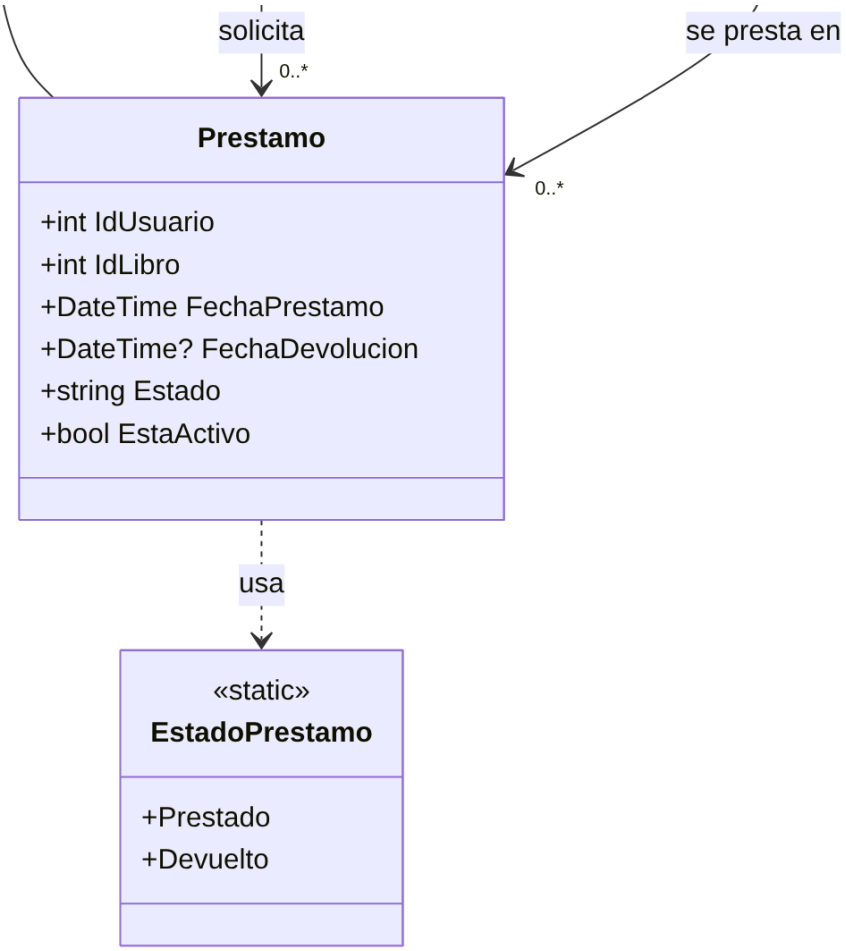
Caso de uso	Descripción	Flujo principal	Flujos alternos
CU-01 Navegar entre módulos	El bibliotecario cambia de módulo desde el menú lateral	Pulsa una opción y el módulo se abre en el panel central	Si el módulo falla al abrir, se muestra un mensaje de error
CU-02 Gestionar libros	Registrar, editar o eliminar libros	Llena el formulario → Guardar → mensaje de éxito → la tabla se actualiza	Dato inválido: advertencia. ISBN repetido: «Ya existe un libro registrado con ese ISBN». Con préstamos: no se elimina
CU-03 Buscar libros	Filtrar por código/ISBN, título, autor o todos los campos	Escribe en *Buscar* y elige el criterio	Sin coincidencias: la tabla queda vacía

Caso de uso	Descripción	Flujo principal	Flujos alternos
CU-04 Gestionar autores	CRUD de autores	Igual a CU-02	Autor con libros: no se elimina
CU-05 Gestionar editoriales	CRUD de editoriales	Igual a CU-02	Editorial con libros: no se elimina
CU-06 Gestionar usuarios	CRUD de usuarios	Igual a CU-02	Documento repetido, correo con formato inválido o usuario con préstamos
CU-07 Registrar préstamo	Prestar un libro a un usuario	Elige usuario, libro y fecha → Guardar → se descuenta un ejemplar	Sin ejemplares, préstamo repetido activo, fecha futura
CU-08 Registrar devolución	Cerrar un préstamo activo	Selecciona el préstamo → fecha → *Registrar devolución* → estado «Devuelto» y +1 disponible	Devolución antes del préstamo o préstamo ya devuelto
CU-09 Consultar y corregir préstamos	Ver, filtrar por estado, corregir fechas o eliminar	Selecciona un préstamo de la tabla	Si estaba activo, al eliminarlo el ejemplar vuelve al inventario
CU-10 Generar reportes	Ver consultas consolidadas	Elige un reporte de la lista	Error de conexión: mensaje claro
CU-11 Exportar reporte a CSV	Guardar el reporte en un archivo	Pulsa *Exportar a CSV* y elige la ruta	Sin datos: advertencia

6. Diagrama de clases

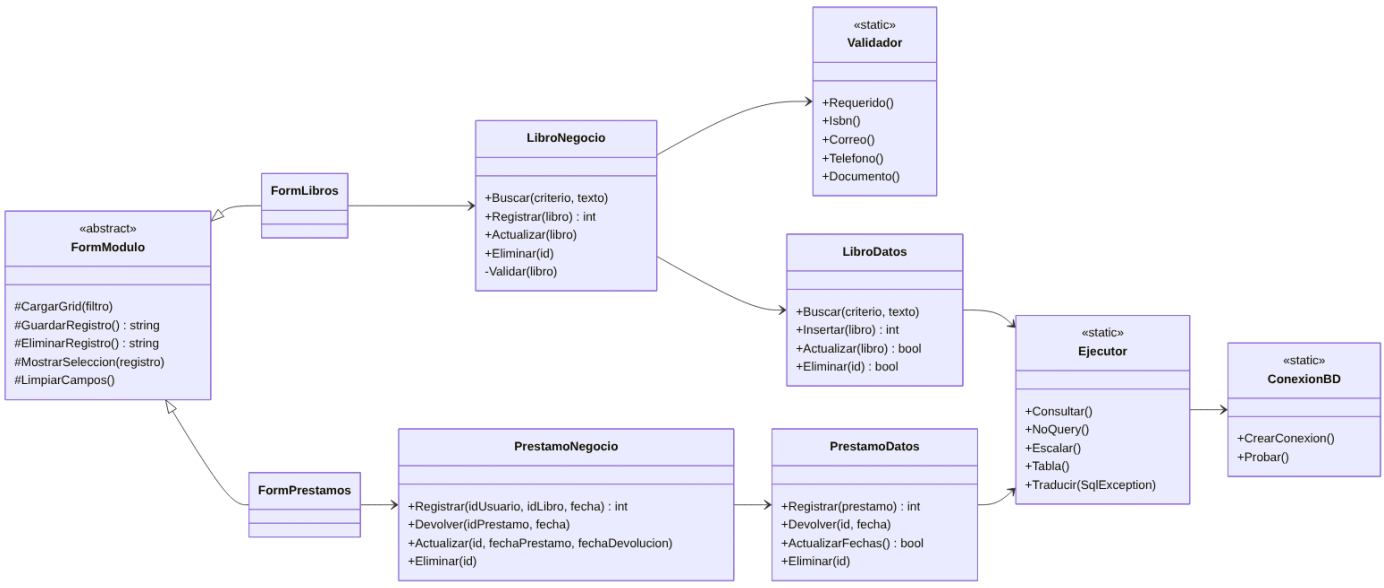
6.1 Entidades del dominio


«abstract»
EntidadBase
+int Id
+bool EsNuevo
«abstract»
Persona
+string Nombre
+string Apellido
+string NombreCompleto
+ToString() : string
Autor
+string Nacionalidad
+ToString() : string
Editorial
+string Nombre
+string Pais
+string Telefono
+ToString() : string
1
escribe
0..*
1
publica
0..*
Libro
+string Isbn
+string Titulo
+int IdAutor
+int IdEditorial
+int AnioPublicacion
+int Existencias
+int Disponibles
+bool HayDisponibilidad
+int Prestados
+string Descripcion
Usuario
+string Documento
+string Telefono
+string Correo
+string Descripcion
1
1



`EntidadBase` concentra el identificador. `Persona` agrupa nombre y apellido, y de ella heredan `Autor` y `Usuario`, que sí comparten esos datos: ahí se justifica la herencia.

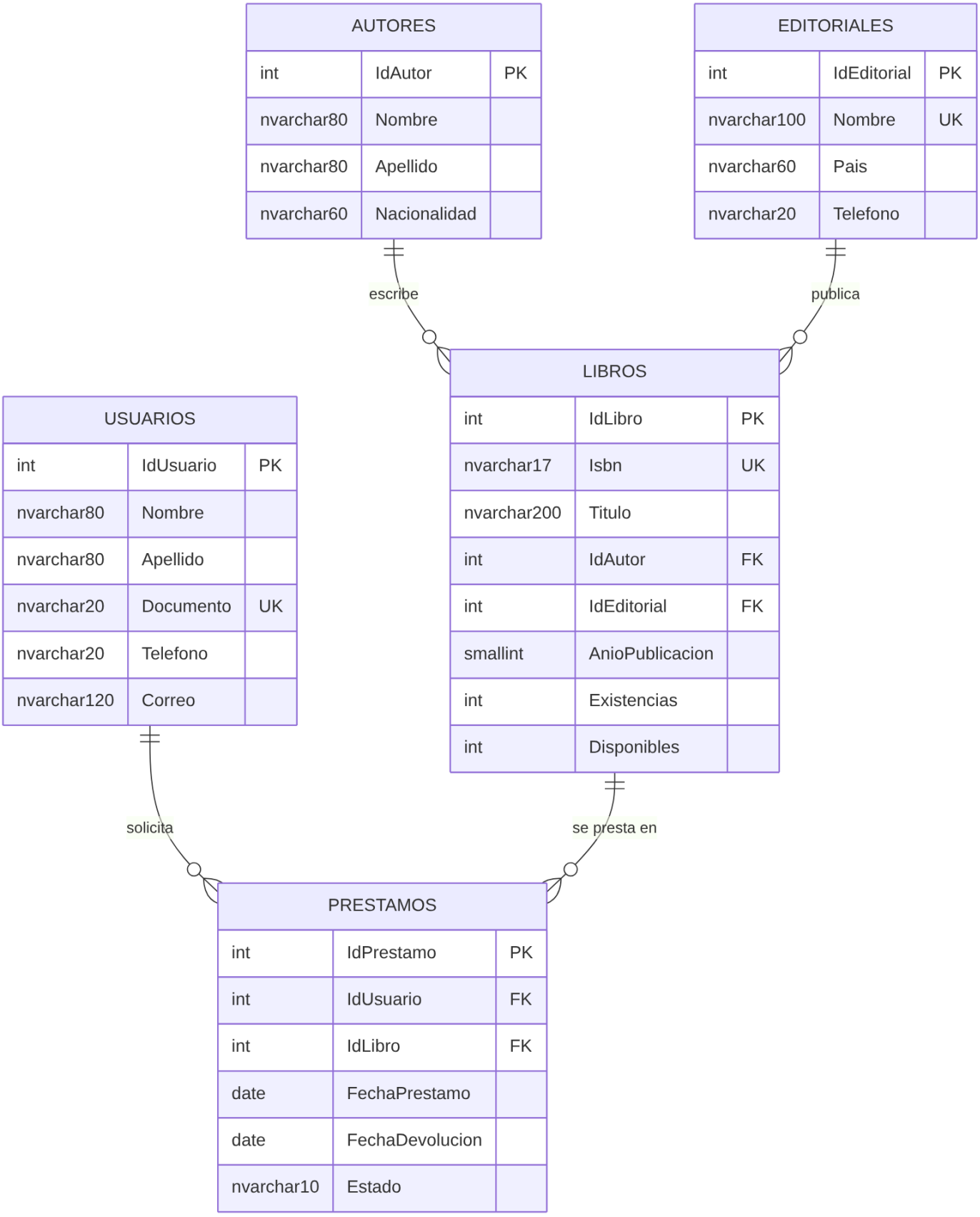
6.2 Relación entre las capas (ejemplo con libros y préstamos)



6.3 Conceptos de POO aplicados

Concepto	Dónde se evidencia
Clases y objetos	`Libro`, `Autor`, `Usuario`, `Editorial`, `Prestamo` y sus instancias en los formularios
Constructores	Cada entidad tiene constructor vacío y constructor con parámetros (`new Autor(id, nombre, apellido, nacionalidad)`)
Encapsulamiento	Propiedades con `get/set`; el `Validador` y `Ejecutor` son internos; los campos de los formularios son `private readonly`
Propiedades y métodos	`Libro.HayDisponibilidad`, `Libro.Prestados`, `Prestamo.EstaActivo`, `Persona.NombreCompleto`
Colecciones	`List<T>` en todas las consultas; `Dictionary<string, Func<DataTable>>` en `ReporteNegocio`
Organización de responsabilidades	Una clase por entidad en cada capa (`*Negocio`, `*Datos`)
Herencia	`EntidadBase → Persona → Autor/Usuario`; `FormEmbebido → FormModulo → FormLibros, FormAutores, ...`
Polimorfismo (bonificación)	`ToString()` sobrescrito en `Autor` (agrega nacionalidad) y `Editorial`; métodos `abstract` de `FormModulo` que cada formulario implementa a su manera

7. Modelo entidad-relación



Relación	Cardinalidad	Clave foránea
Autor → Libro	Un autor escribe muchos libros; cada libro tiene un autor	`Libros.IdAutor`
Editorial → Libro	Una editorial publica muchos libros	`Libros.IdEditorial`
Usuario → Préstamo	Un usuario puede tener muchos préstamos	`Prestamos.IdUsuario`
Libro → Préstamo	Un libro puede aparecer en muchos préstamos	`Prestamos.IdLibro`

No se creó una tabla `DetallePrestamos` : cada préstamo corresponde a un solo libro, por lo que el modelo no la necesita.

8. Diccionario de datos

Base de datos: **BibliotecaDB** (SQL Server, intercalación `Latin1_General_CI_AI` , sin distinción de mayúsculas ni tildes).

8.1 Autores

Campo	Tipo	Nulo	Clave / restricción	Descripción
IdAutor	INT IDENTITY	No	PK	Identificador del autor
Nombre	NVARCHAR(80)	No	CHECK no vacío	Nombre(s)
Apellido	NVARCHAR(80)	No	CHECK no vacío; UNIQUE (Nombre, Apellido)	Apellido(s)
Nacionalidad	NVARCHAR(60)	Sí	—	País de origen

8.2 Editoriales

Campo	Tipo	Nulo	Clave / restricción	Descripción
IdEditorial	INT IDENTITY	No	PK	Identificador único
Nombre	NVARCHAR(100)	No	UNIQUE; CHECK no vacío	Nombre de la editorial
Pais	NVARCHAR(60)	Sí	—	País

Campo	Tipo	Nulo	Clave / restricción	Descripción
Telefono	NVARCHAR(20)	Sí	—	Teléfono de contacto

8.3 Usuarios

Campo	Tipo	Nulo	Clave / restricción	Descripción
IdUsuario	INT IDENTITY	No	PK	Identificador del usuario
Nombre	NVARCHAR(80)	No	CHECK no vacío	Nombre(s)
Apellido	NVARCHAR(80)	No	CHECK no vacío	Apellido(s)
Documento	NVARCHAR(20)	No	UNIQUE	Documento de identidad
Telefono	NVARCHAR(20)	Sí	—	Teléfono
Correo	NVARCHAR(120)	Sí	CHECK formato `%_@_%._%`	Correo electrónico

8.4 Libros

Campo	Tipo	Nulo	Clave / restricción	Descripción
IdLibro	INT IDENTITY	No	PK	Identificador del libro
Isbn	NVARCHAR(17)	No	UNIQUE	Código ISBN
Titulo	NVARCHAR(200)	No	CHECK no vacío	Título
IdAutor	INT	No	FK → Autores	Autor del libro
IdEditorial	INT	No	FK → Editoriales	Editorial
AnioPublicacion	SMALLINT	No	CHECK entre 1000 y 2100	Año de publicación
Existencias	INT	No	CHECK ≥ 0; DEFAULT 1	Total de ejemplares de la biblioteca
Disponibles	INT	No	CHECK ≥ 0 y ≤ Existencias; DEFAULT 1	Ejemplares en el estante

8.5 Préstamos

Campo	Tipo	Nulo	Clave / restricción	Descripción
IdPrestamo	INT IDENTITY	No	PK	Identificador del préstamo
IdUsuario	INT	No	FK → Usuarios	Quién lo solicita
IdLibro	INT	No	FK → Libros	Libro prestado
FechaPrestamo	DATE	No	DEFAULT hoy	Fecha en que se presta
FechaDevolucion	DATE	Sí	CHECK de coherencia	Fecha en que se devuelve
Estado	NVARCHAR(10)	No	CHECK `Prestado` / `Devuelto`; DEFAULT `Prestado`	Estado del préstamo

Restricciones adicionales de `Prestamos`: `CK_Prestamos_Coherencia` (si está *Prestado* no hay fecha de devolución; si está *Devuelto* hay fecha y no es anterior al préstamo) y el índice único filtrado `UX_Prestamos_Activo (IdUsuario, IdLibro) WHERE Estado = 'Prestado'`.

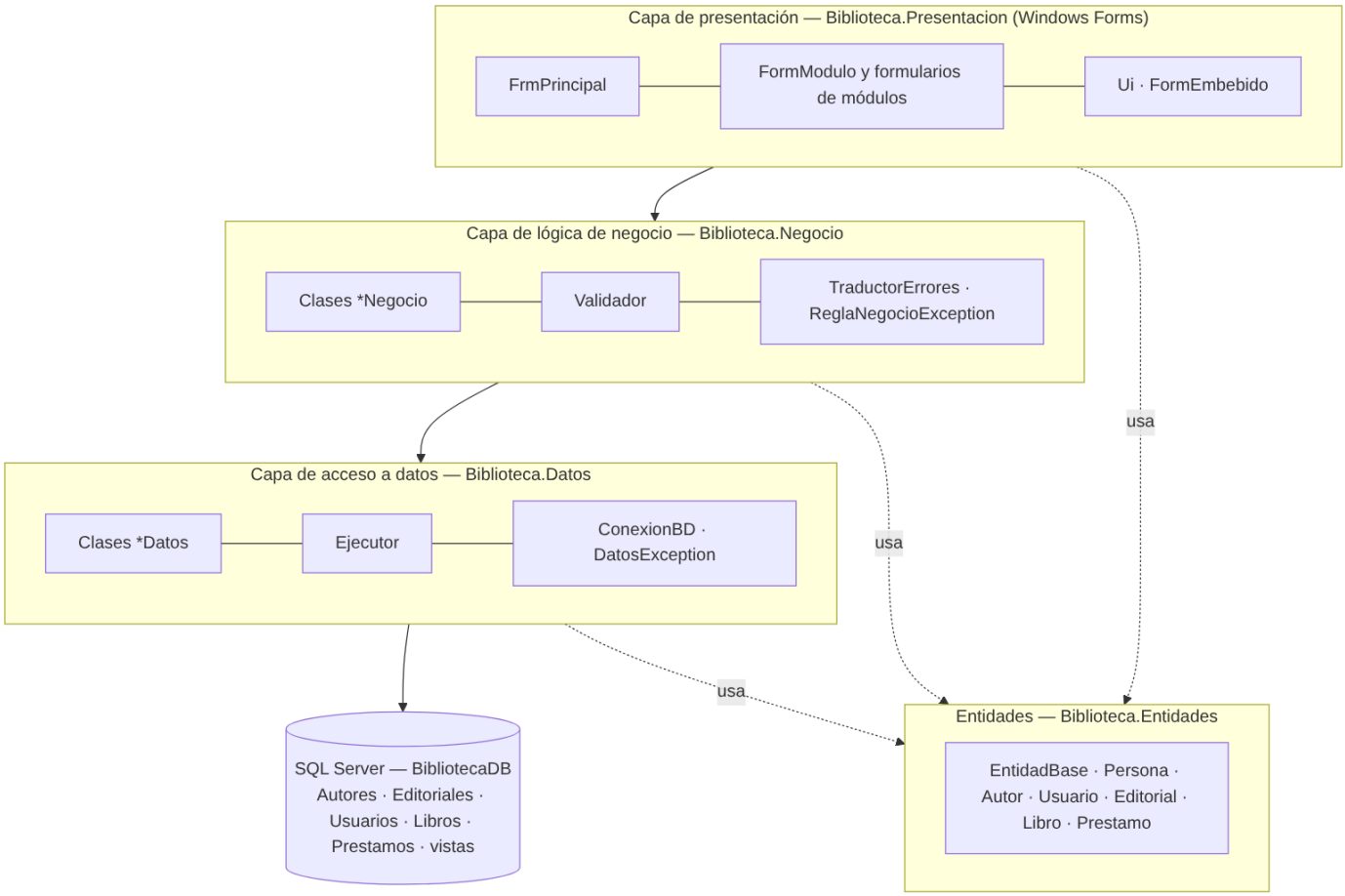
8.6 Vistas e índices

- `vw_Libros`: libros con el nombre del autor y de la editorial.
- `vw_Prestamos`: préstamos con usuario, libro y días transcurridos.
- Índices de apoyo: `IX_Libros_IdAutor`, `IX_Libros_IdEditorial`, `IX_Libros_Titulo`, `IX_Prestamos_IdUsuario`, `IX_Prestamos_IdLibro`.

El script completo está en `database/biblioteca_script.sql` e incluye datos de prueba (10 autores, 6 editoriales, 10 usuarios, 14 libros y 12 préstamos).

9. Arquitectura del sistema

La solución se divide en cuatro proyectos de Visual Studio con dependencias en un solo sentido: la presentación llama a negocio, negocio llama a datos y datos habla con SQL Server.



Capa	Proyecto	Responsabilidad	Clases principales
Presentación	`Biblioteca.Presentacion`	Formularios, controles, captura de datos y mensajes	`FrmPrincipal`, `FormModulo`, `FormLibros`, `FormPrestamos`, `FormReportes`, `Ui`
Lógica de negocio	`Biblioteca.Negocio`	Reglas del sistema, validaciones y coordinación	`LibroNegocio`, `PrestamoNegocio`, `Validador`, `TraductorErrores`
Acceso a datos	`Biblioteca.Datos`	Conexión, consultas parametrizadas y	`ConexionBD`, `Ejecutor`, `LibroDatos`,

Capa	Proyecto	Responsabilidad	Clases principales
		transacciones	`PrestamoDatos`, `ReporteDatos`
Entidades	`Biblioteca.Entidades`	Modelo de objetos compartido por las capas	`Libro`, `Autor`, `Usuario`, `Editorial`, `Prestamo`
Base de datos	SQL Server	Almacenamiento, relaciones y restricciones	`BibliotecaDB`

Estructura de carpetas del repositorio

```
BibliotecaApp/
├── BibliotecaApp.sln
├── README.md
├── database/biblioteca_script.sql
├── docs/            Documento_Tecnico.md y diagramas/
├── capturas/        capturas de pantalla del sistema
└── src/
    ├── Biblioteca.Entidades/
    ├── Biblioteca.Datos/
    ├── Biblioteca.Negocio/
    └── Biblioteca.Presentacion/
```

Manejo de errores entre capas. SQL Server lanza `SqlException`; `Ejecutor.Traducir` la convierte en `DatosException` (duplicado, restricción, conexión, etc.); la capa de negocio la transforma en `ReglaNegocioException` con un mensaje para el usuario; y el método `Seguro()` de los formularios la muestra como advertencia. Así la interfaz nunca depende de detalles de SQL Server.

10. Explicación de los módulos desarrollados

10.1 Inicio y navegación (RF08)

`FrmPrincipal` tiene un menú lateral (Inicio, Libros, Usuarios, Autores, Editoriales, Préstamos, Reportes y Salir). Cada opción abre su módulo dentro del panel central y cierra el anterior para

liberar recursos. `FormInicio` muestra tarjetas con las cifras principales (títulos, ejemplares disponibles, autores, editoriales, usuarios y préstamos activos) y una guía rápida.

10.2 Plantilla de los módulos de gestión

`FormModulo` dibuja la pantalla de gestión: campos a la izquierda; buscador y tabla a la derecha; botones *Nuevo*, *Guardar* y *Eliminar*. Cada módulo solo define sus campos y cómo guardar, eliminar o mostrar un registro. Esto evita repetir código y mantiene la interfaz uniforme.

10.3 Libros (RF01)

Campos: ISBN, título, autor y editorial (listas desplegables con registros existentes), año y existencias. Buscador por *todos los campos*, *código/ISBN*, *título* o *autor*. La tabla muestra además los ejemplares disponibles. Al editar existencias, la base conserva los ejemplares ya prestados (`Disponibles = ExistenciasNuevas – Prestados`).

10.4 Autores y editoriales (RF02, RF03)

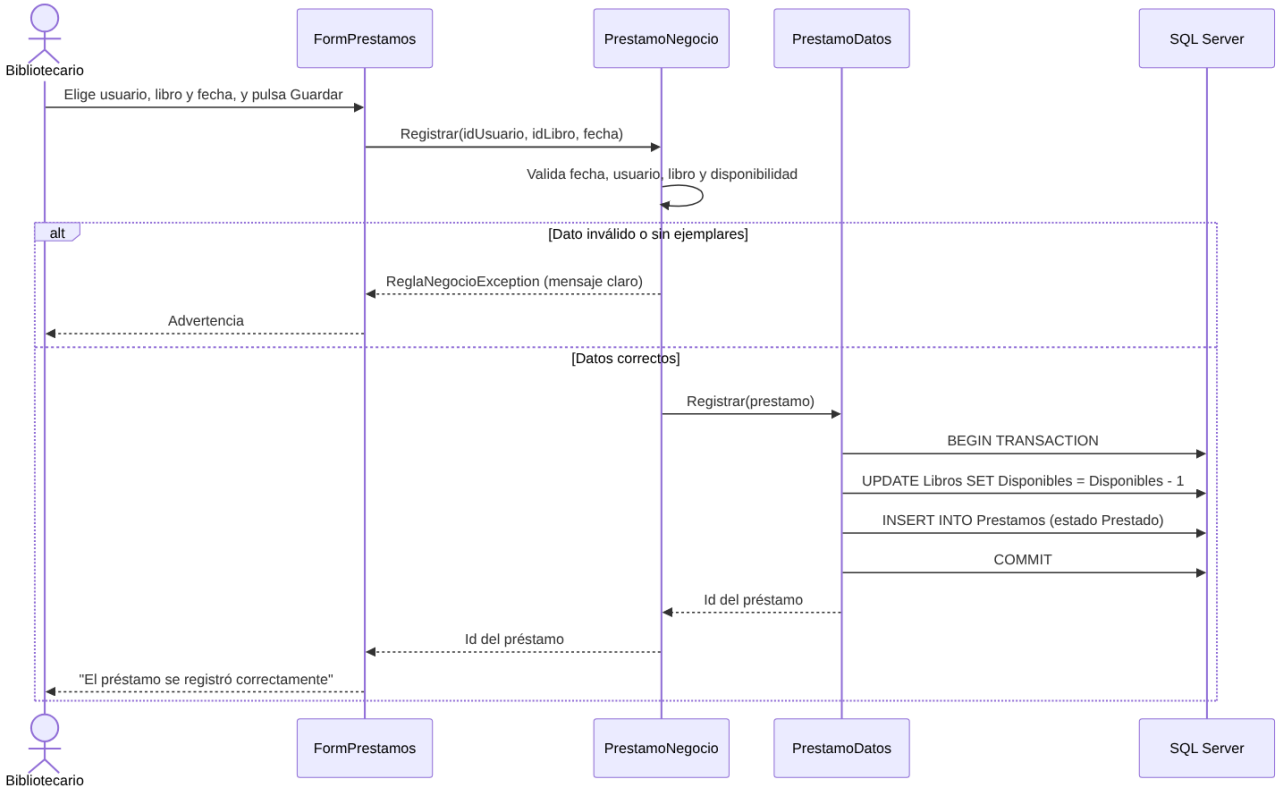
CRUD con validación de campos obligatorios y longitudes. No se permite repetir nombre y apellido de autor ni nombre de editorial, ni eliminar registros que tengan libros asociados.

10.5 Usuarios (RF04)

CRUD con validación de documento (5 a 20 letras o números, único), teléfono (solo números, +, espacios y guiones) y correo con formato válido. No se elimina un usuario con préstamos.

10.6 Préstamos y devoluciones (RF05, RF06)

Para un préstamo nuevo se elige usuario, libro y fecha. El sistema verifica que existan, que haya ejemplares y que el usuario no tenga ya ese libro pendiente. El préstamo y el descuento del ejemplar se hacen en **una sola transacción**. Al seleccionar un préstamo existente se puede corregir la fecha, registrar la devolución (cambia el estado a *Devuelto*, guarda la fecha y devuelve el ejemplar al inventario) o eliminarlo. La tabla permite filtrar por estado y colorea *Prestado* y *Devuelto*.



10.7 Reportes (RF07)

Diez consultas disponibles en una lista: resumen general, listado de libros, de autores, de editoriales, de usuarios, relación de préstamos, préstamos activos, préstamos devueltos, historial de préstamos por usuario y libros más prestados. Cualquier resultado se puede exportar a **CSV** (separador `;`, UTF-8) para abrirlo en Excel.

11. Conexión con SQL Server

1. **Cadena de conexión.** Está en `src/Biblioteca.Presentacion/App.config`, entrada `BibliotecaDB`:

```
<add name="BibliotecaDB"
     connectionString="Server=localhost\SQLEXPRESS;Database=BibliotecaDB;Trusted
     providerName="Microsoft.Data.SqlClient" />
```

   Se puede cambiar el servidor sin recompilar.
2. **Proveedor.** Se usa el paquete `Microsoft.Data.SqlClient` y `System.Configuration.ConfigurationManager` para leer el archivo de configuración.
3. **Clase `ConexionBD`.** Lee la cadena, crea las conexiones (`CrearConexion`) y ofrece `Probar` para verificar que SQL Server responde.

4. **Clase `Ejecutor`.** Es el único punto por el que pasan las consultas. Abre y cierra la conexión con `using`, agrega siempre los valores como `SqlParameter` (nunca concatena texto del usuario, lo que evita la inyección SQL) y traduce los errores de SQL Server.
5. **Transacciones.** `PrestamoDatos.Registrar`, `Devolver` y `Eliminar` usan `BeginTransaction` para que el préstamo y el inventario cambien juntos o no cambien.
6. **Verificación al iniciar.** `Program.Main` prueba la conexión antes de abrir la ventana principal; si falla, muestra qué revisar (servicio, script SQL, cadena de conexión).

12. Capturas de pantalla del sistema

> Las imágenes están en la carpeta `capturas/` del repositorio. Cada una se guarda con el nombre indicado.

Figura 1. Pantalla de inicio



Figura 2. Módulo de libros

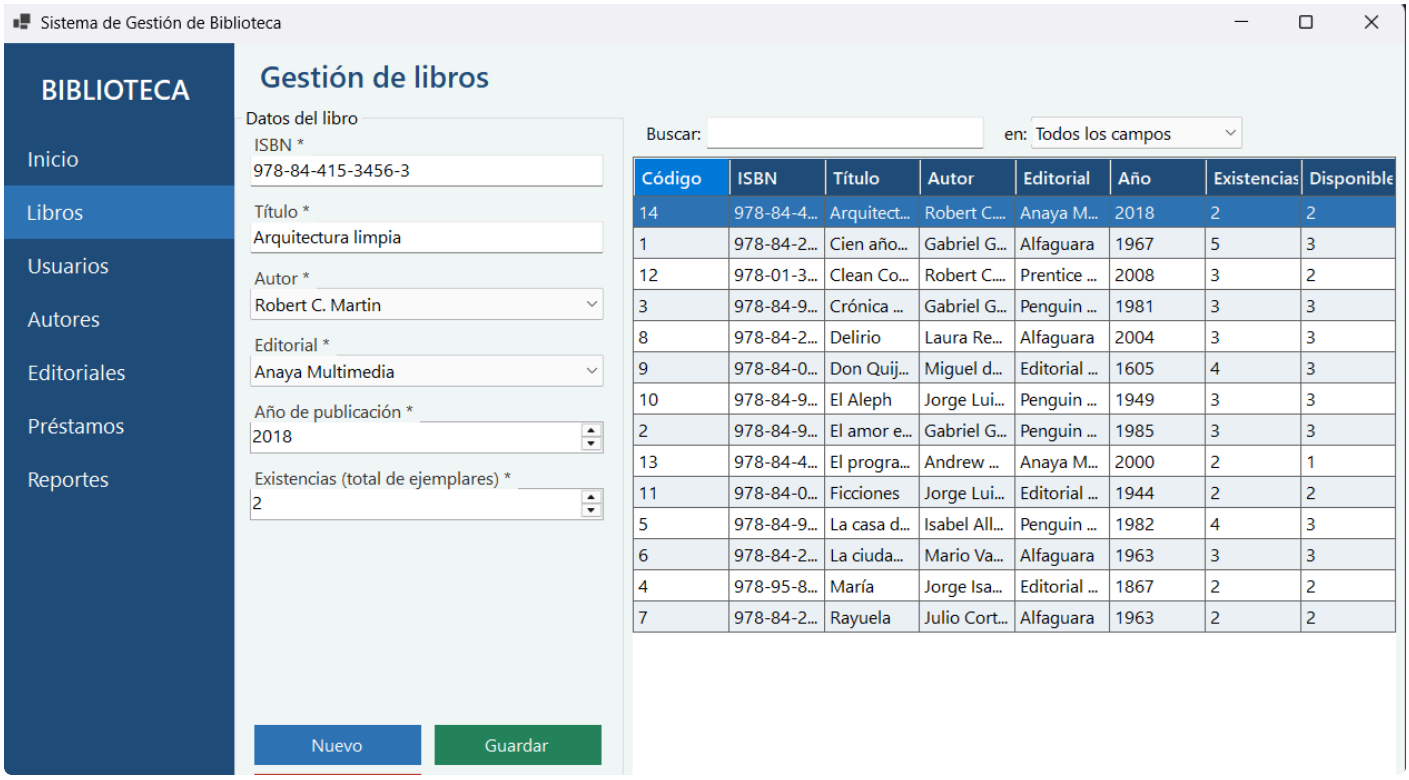


Figura 3. Búsqueda de libros por título o autor

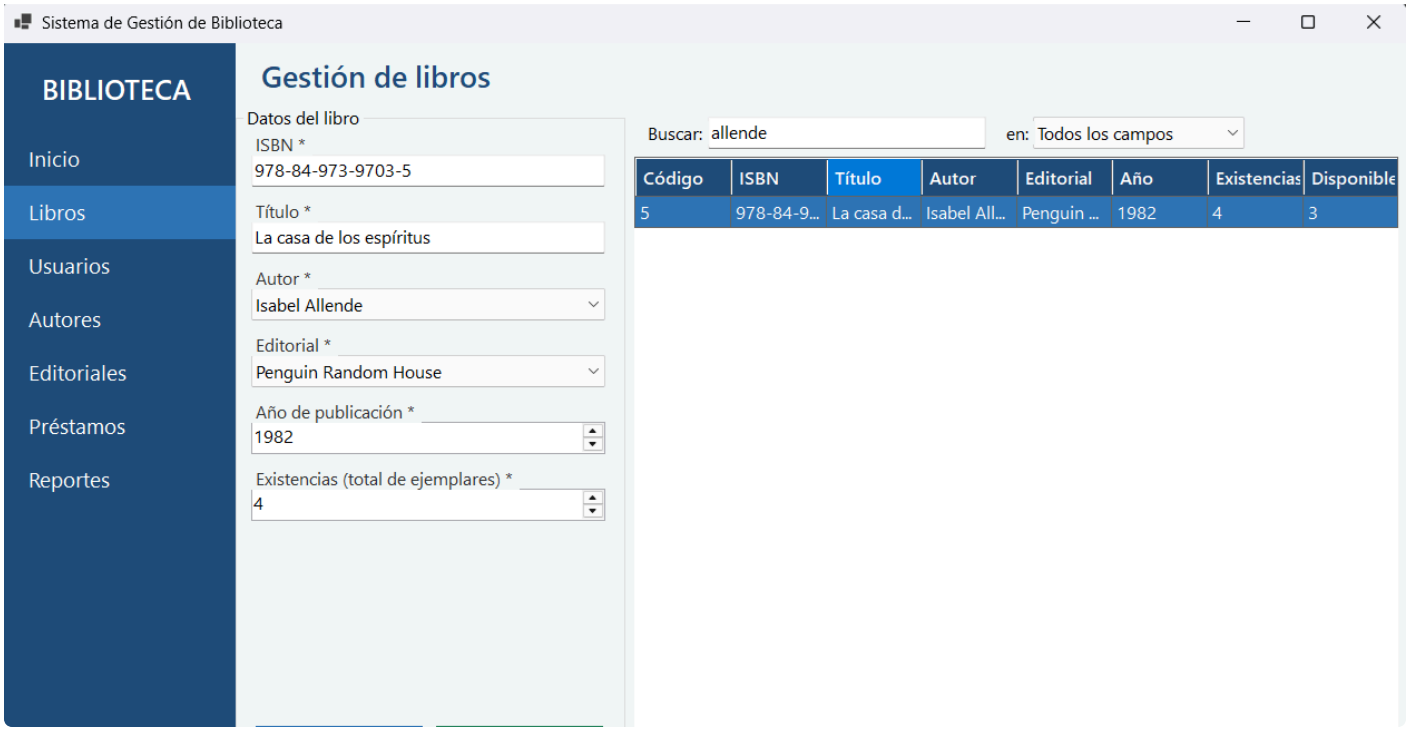


Figura 4. Validación al guardar un libro con datos incorrectos

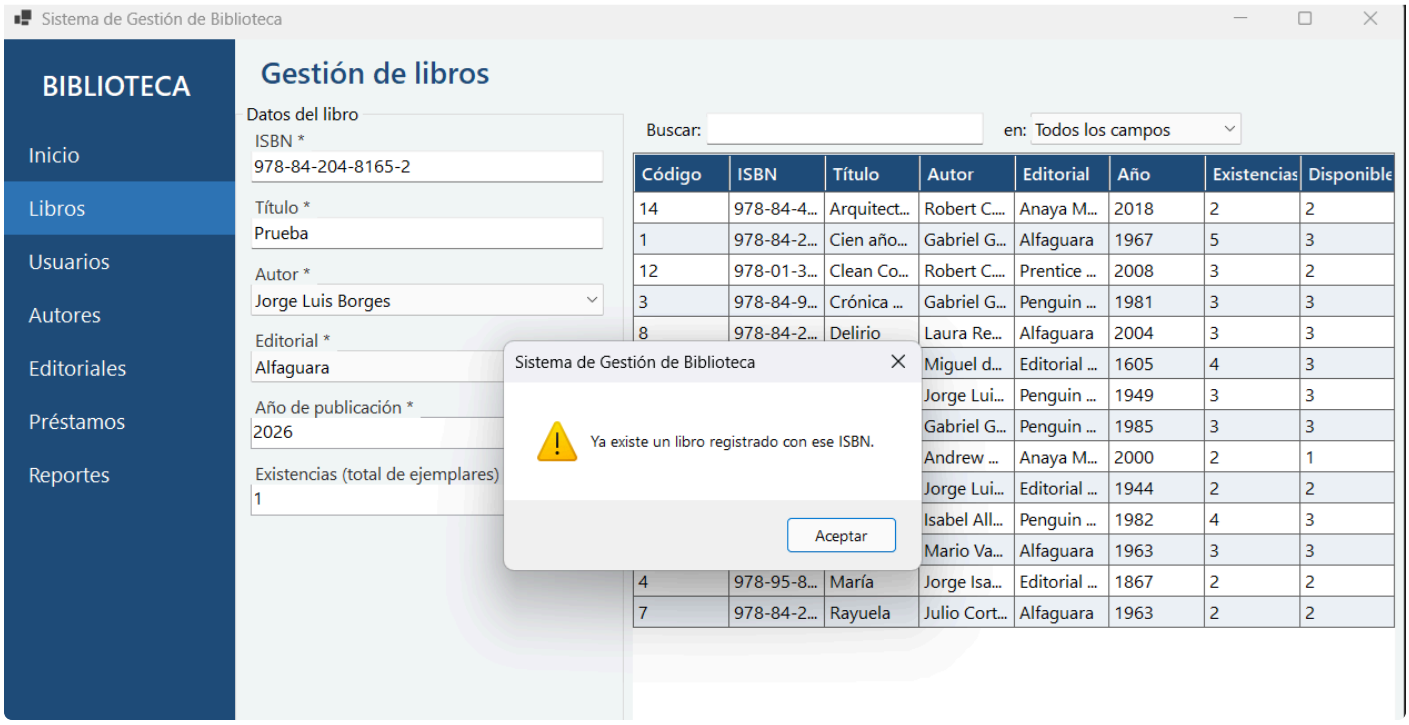


Figura 5. Módulo de autores



Figura 6. Módulo de editoriales

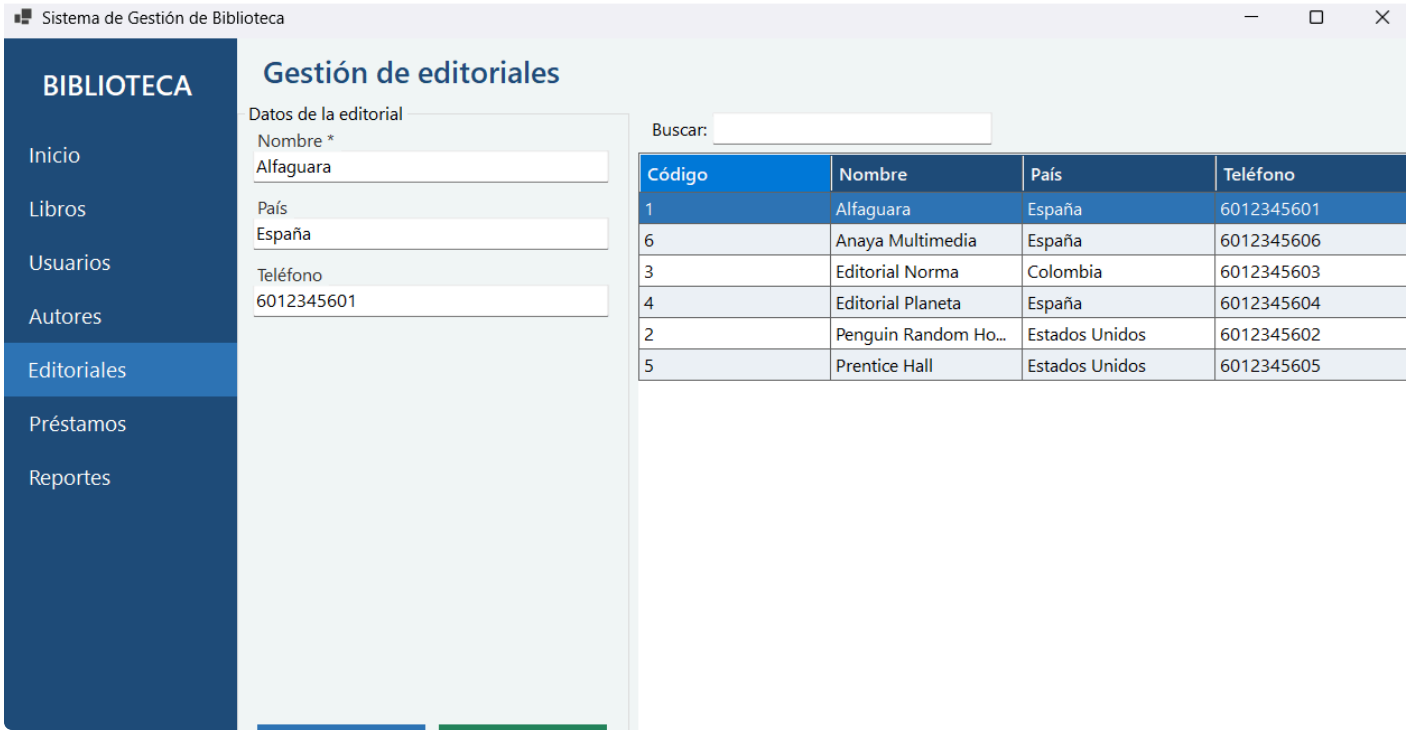


Figura 7. Módulo de usuarios

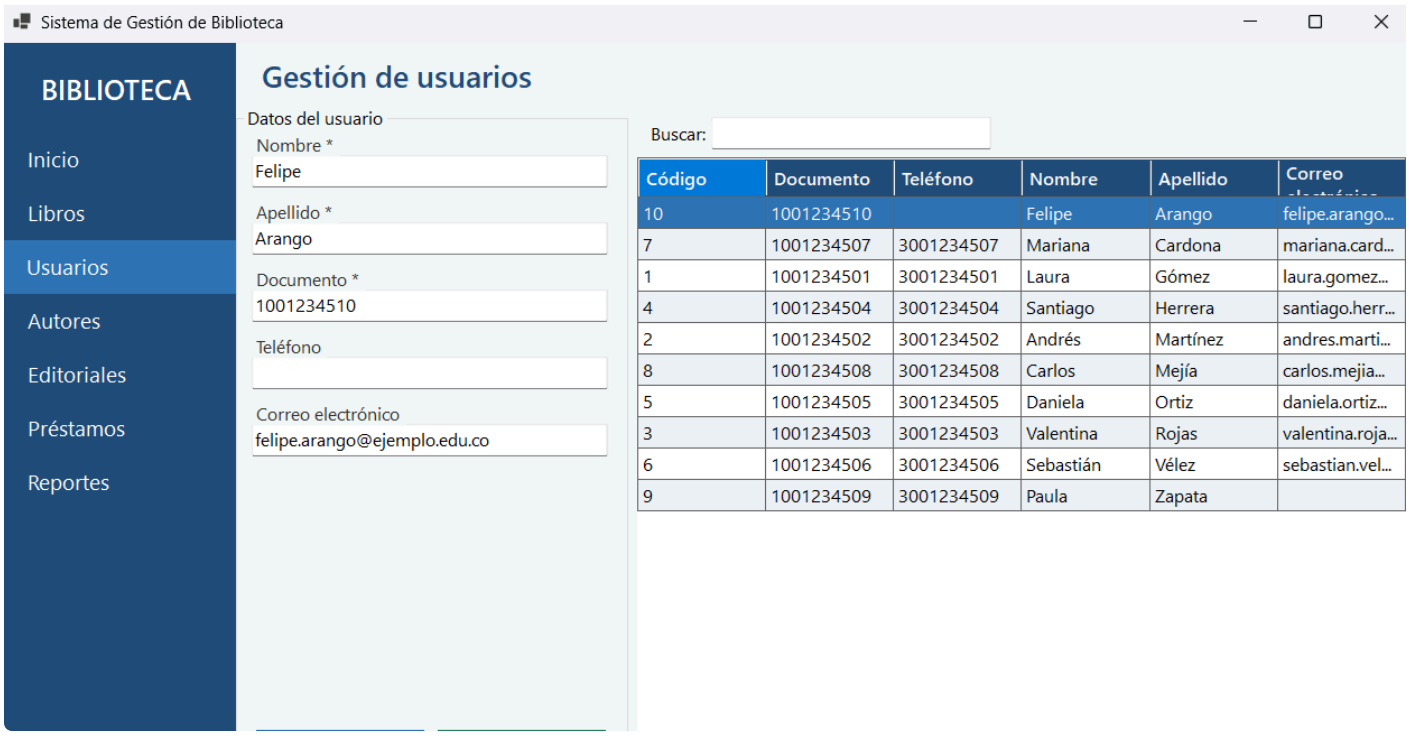


Figura 8. Validación de usuario (correo o documento inválido)

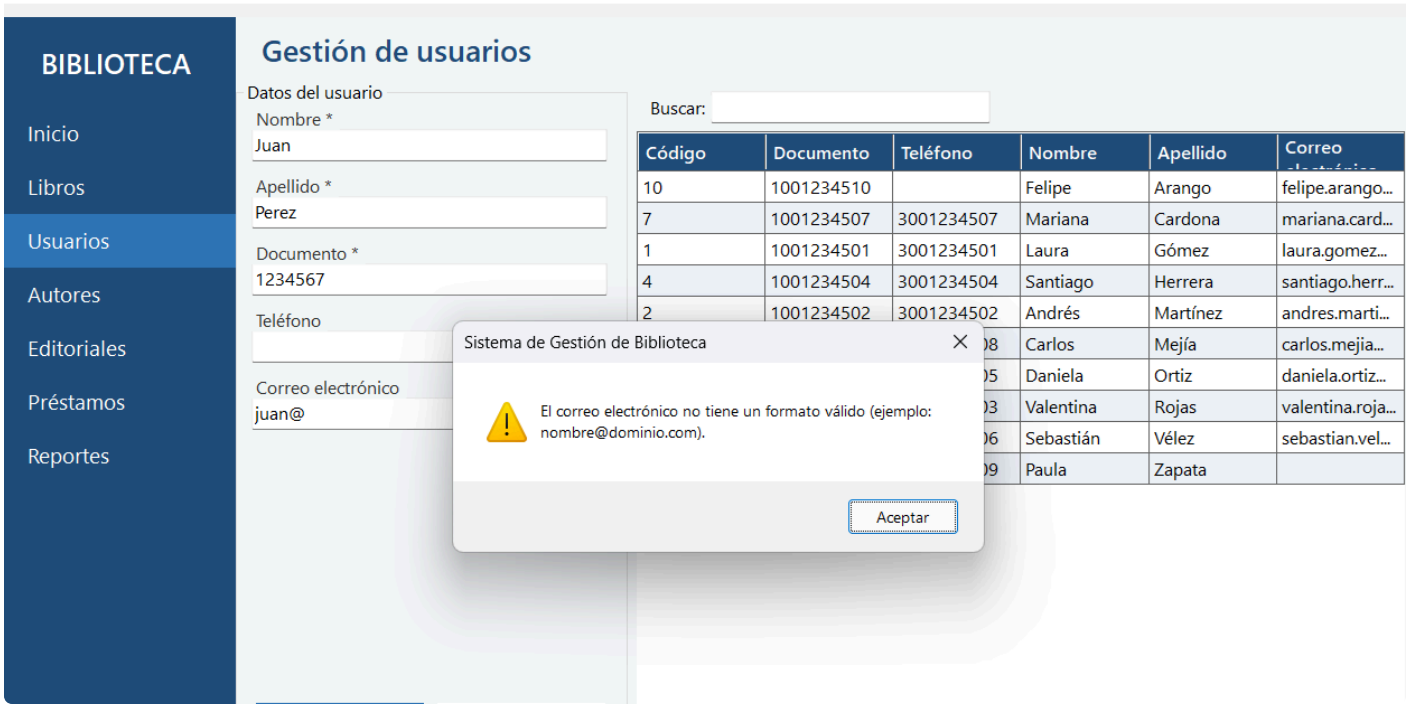


Figura 9. Módulo de préstamos

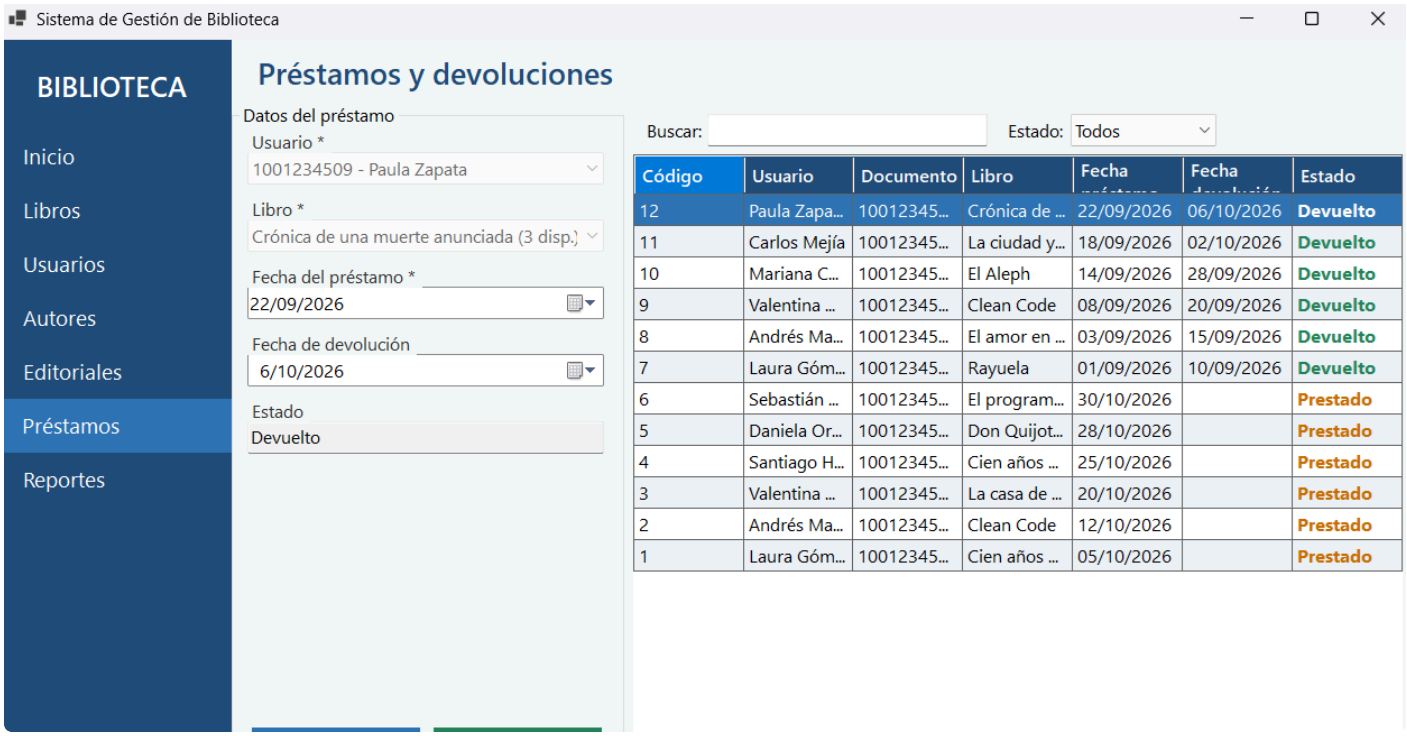


Figura 10. Préstamo registrado correctamente

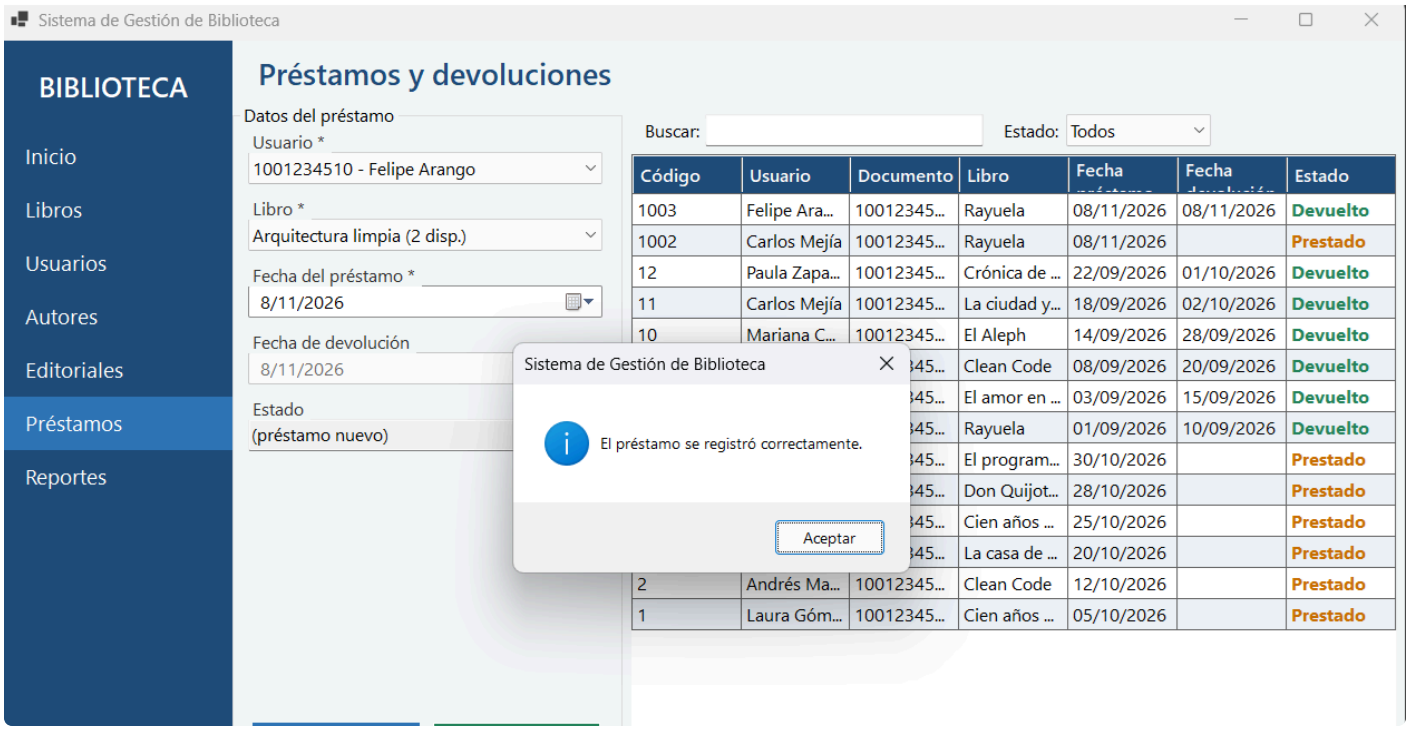


Figura 11. Devolución registrada

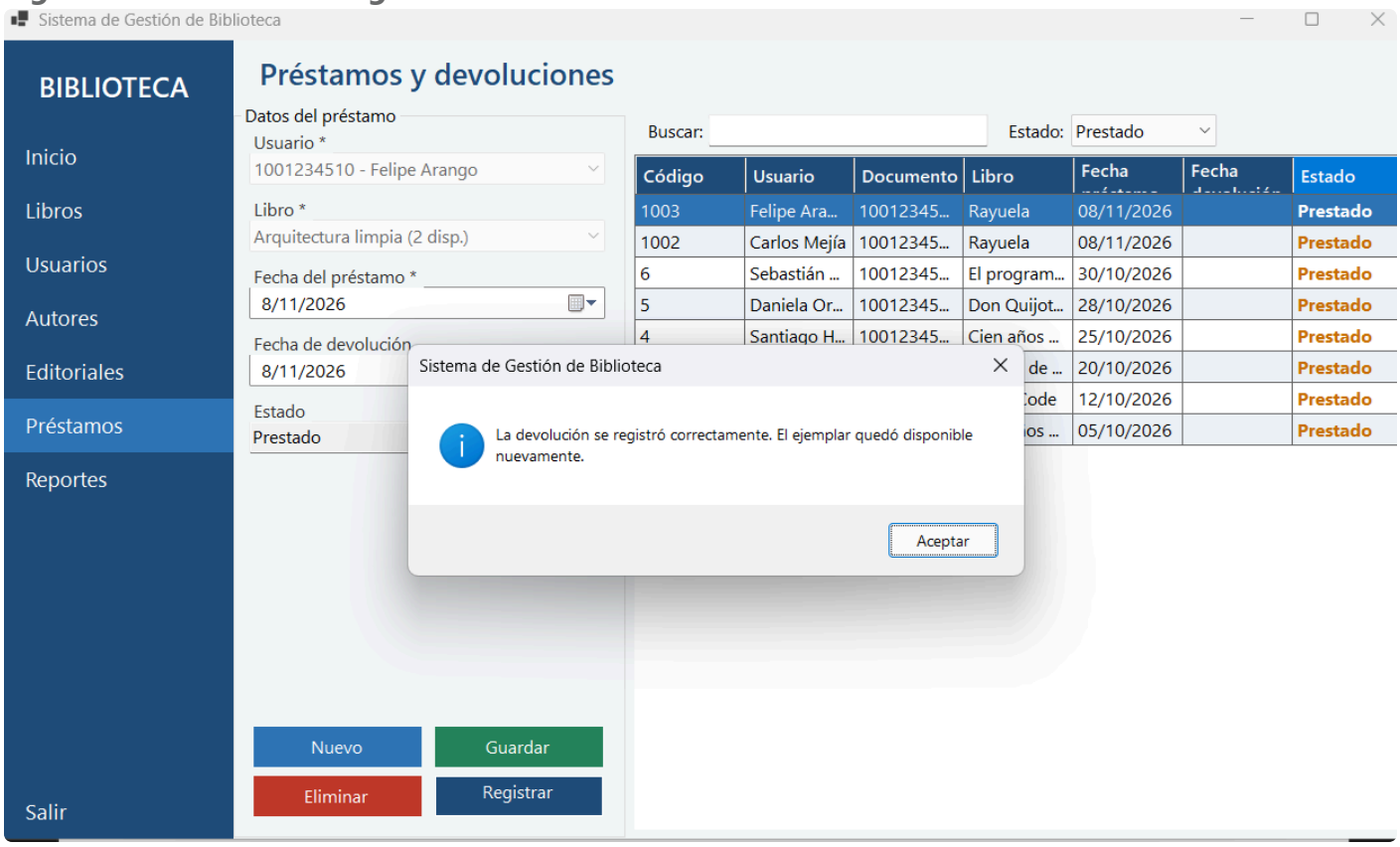


Figura 12. Mensaje al intentar prestar un libro sin ejemplares



Figura 13. Mensaje al eliminar un autor con libros

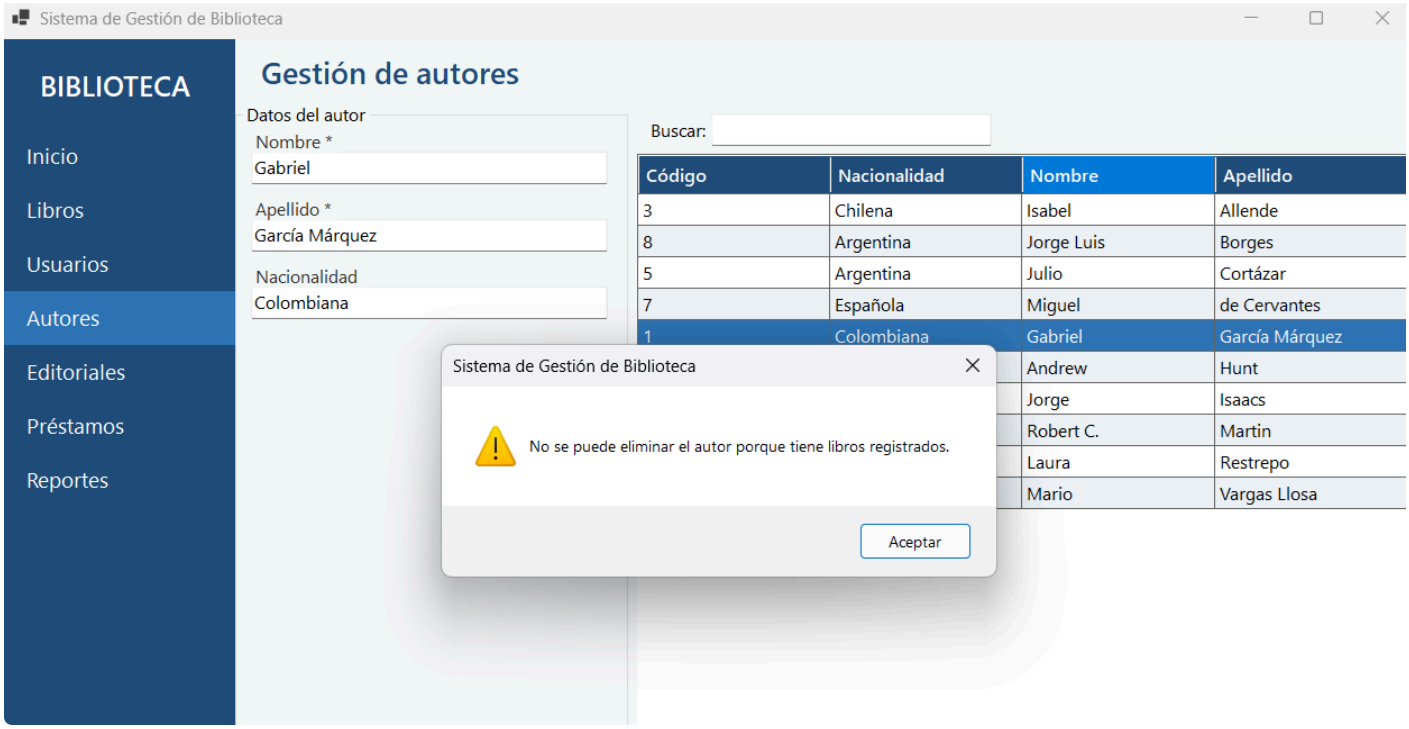


Figura 14. Módulo de reportes



Figura 15. Base de datos y tablas en SQL Server Management Studio

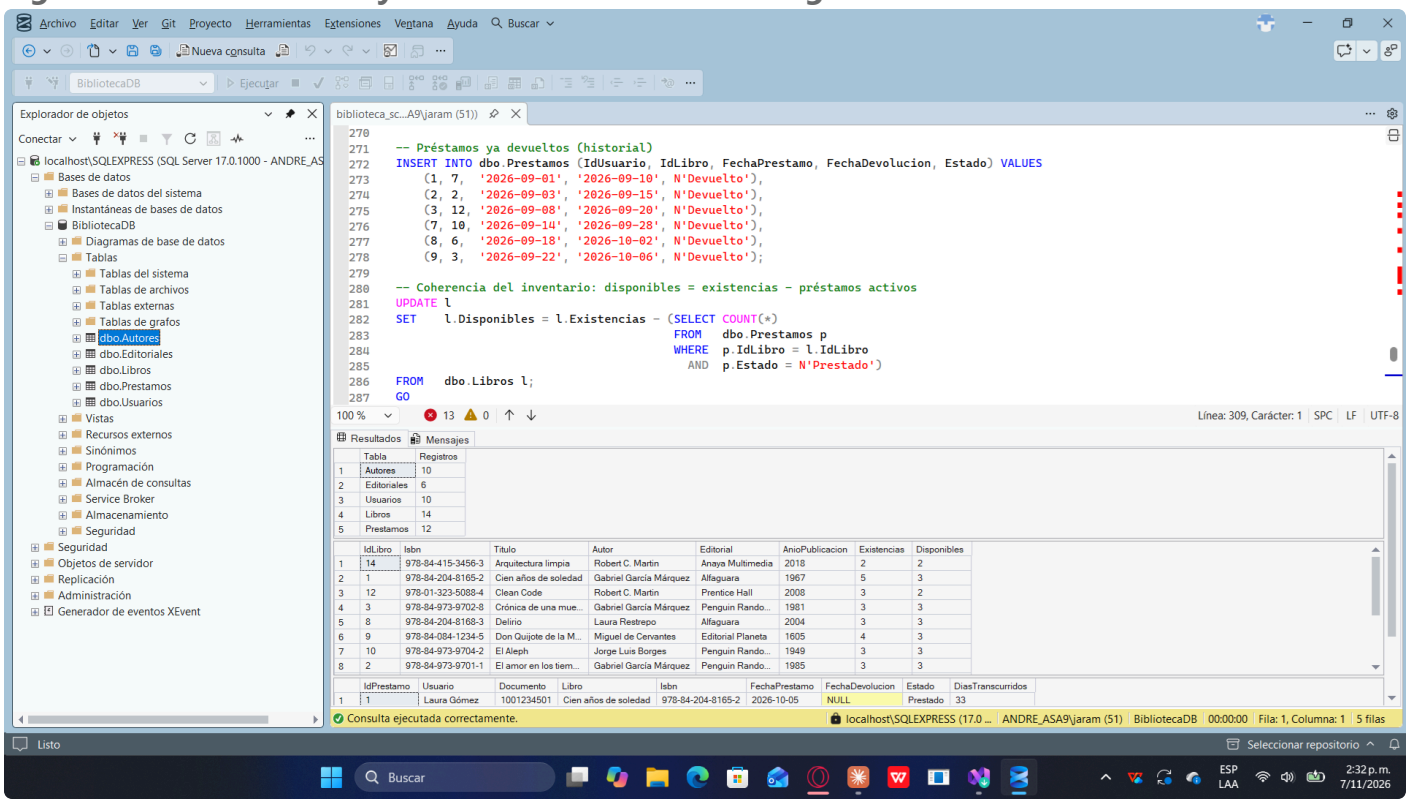


Figura 16. Repositorio en GitHub

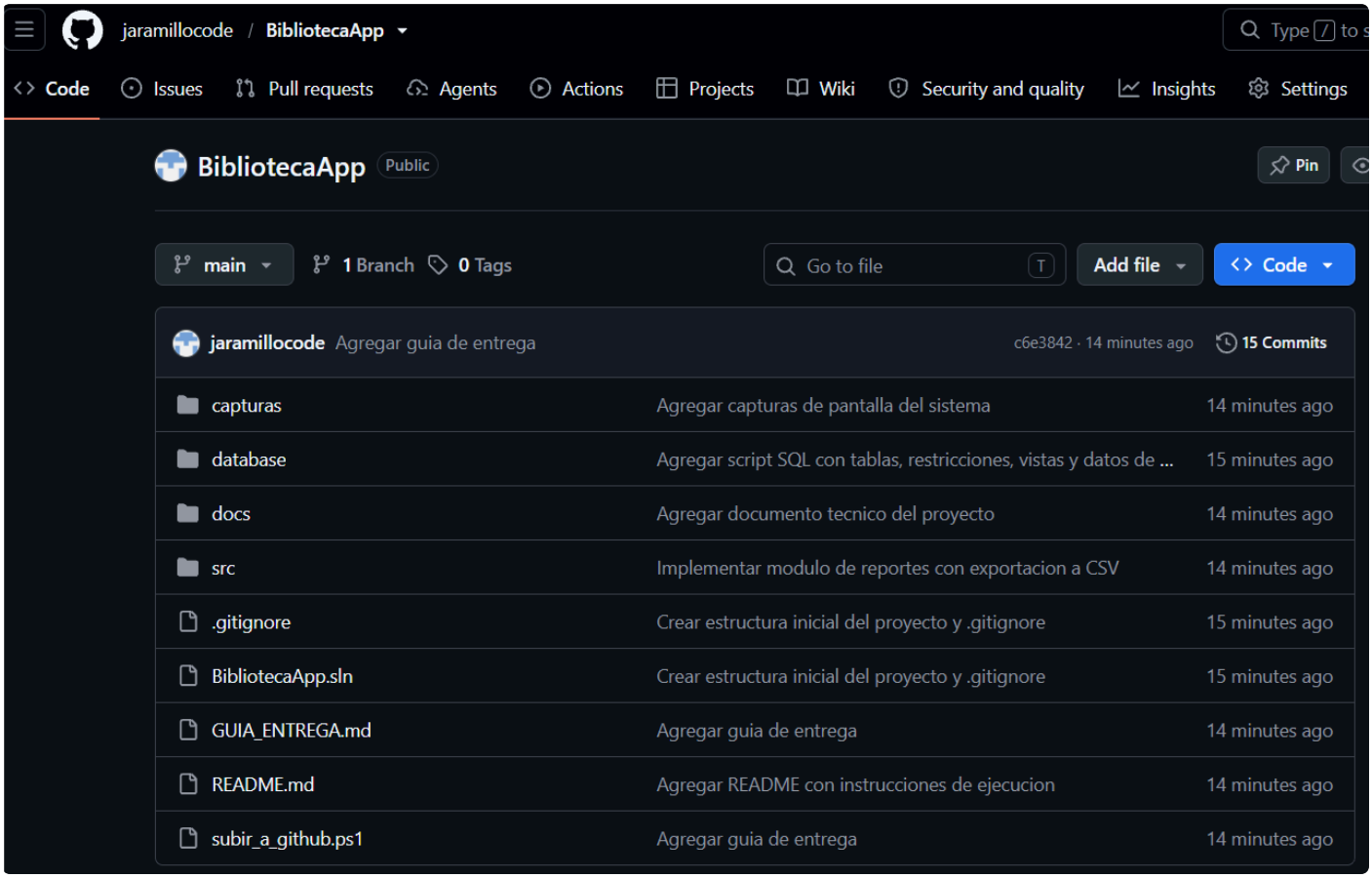
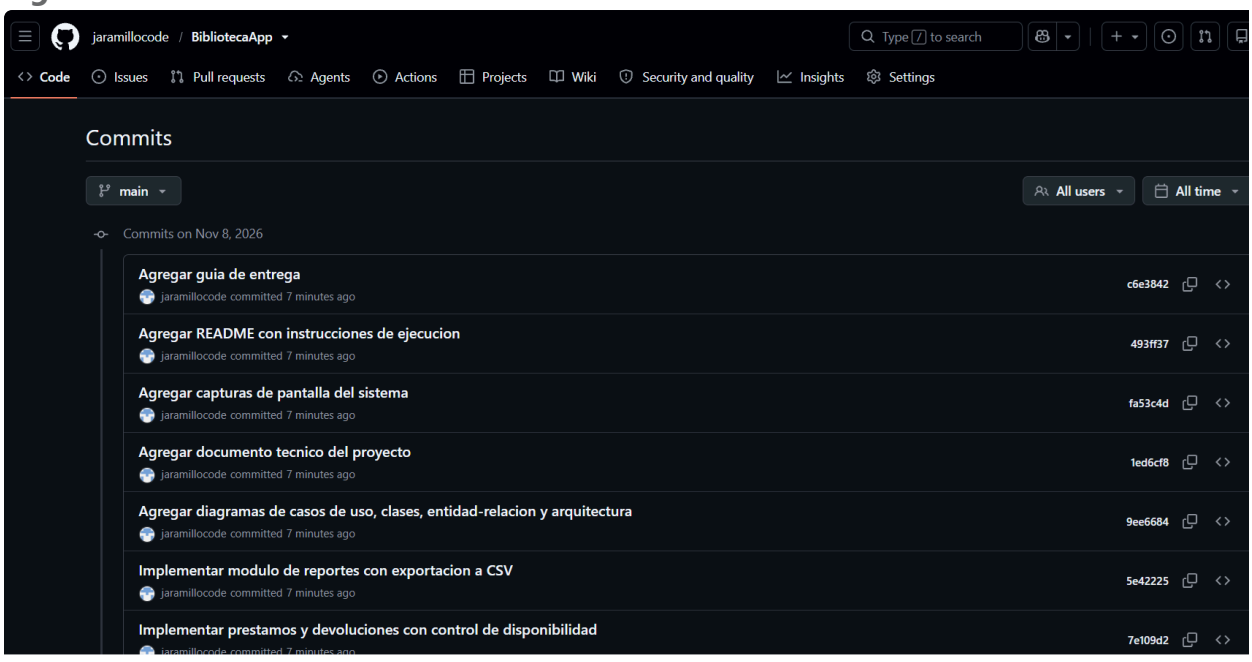


Figura 17. Historial de commits



13. Pruebas de funcionamiento

Las pruebas se hicieron sobre la base cargada con el script SQL. En la columna **Resultado** se marca el estado observado al ejecutar la aplicación.

N.º	Módulo	Prueba	Datos de entrada	Resultado esperado	Resultado
1	Conexión	Iniciar con SQL Server activo	—	Se abre la ventana principal	✓ Aprobada
2	Conexión	Iniciar con servidor incorrecto en `App.config`	Server= `noexiste`	Mensaje «No se pudo conectar con la base de datos»	✓ Aprobada
3	Navegación	Pasar por todos los módulos	Clic en cada opción	Cada módulo se abre en el panel central	✓ Aprobada
4	Autores	Registrar autor	Nombre `Pablo`, Apellido `Neruda`	«El autor se registró correctamente» y aparece en la tabla	✓ Aprobada
5	Autores	Registrar autor sin apellido	Apellido vacío	«El apellido es obligatorio.»	✓ Aprobada
6	Autores	Registrar autor repetido	`Isabel Allende`	«Ya existe un autor con ese nombre y apellido.»	✓ Aprobada
7	Autores	Eliminar autor con libros	Seleccionar `Gabriel García Márquez`	«No se puede eliminar el autor porque tiene libros registrados.»	✓ Aprobada
8	Editoriales	Editar una editorial	Cambiar el país de `Editorial Norma`	«La editorial se actualizó correctamente»	✓ Aprobada

N.º	Módulo	Prueba	Datos de entrada	Resultado esperado	Resultado
9	Libros	Registrar libro	ISBN `978-84-204-9999-0`, título, autor, editorial, año 2020, existencias 2	«El libro se registró correctamente», disponibles = 2	✓ Aprobada
10	Libros	ISBN repetido	ISBN `978-84-204-8165-2`	«Ya existe un libro registrado con ese ISBN.»	✓ Aprobada
11	Libros	ISBN con formato inválido	`ABC`	Mensaje de formato de ISBN	✓ Aprobada
12	Libros	Buscar por autor	Criterio *Autor*, texto `allende`	Solo aparecen libros de Isabel Allende	✓ Aprobada
13	Libros	Bajar existencias por debajo de lo prestado	`Cien años de soledad` (2 prestados) con existencias 1	Mensaje de que hay 2 ejemplares prestados	✓ Aprobada
14	Usuarios	Registrar usuario	Datos completos y documento nuevo	«El usuario se registró correctamente»	✓ Aprobada
15	Usuarios	Correo inválido	`juan@`	«El correo electrónico no tiene un formato válido»	✓ Aprobada
16	Usuarios	Documento repetido	`1001234501`	«Ya existe un usuario con ese número de documento.»	✓ Aprobada
17	Usuarios	Eliminar usuario con préstamos	`Laura Gómez`	«No se puede eliminar el usuario porque tiene	✓ Aprobada

N.º	Módulo	Prueba	Datos de entrada	Resultado esperado	Resultado
				préstamos registrados.»	
18	Préstamos	Registrar préstamo	Usuario `Paula Zapata`, libro `Rayuela`, fecha de hoy	Préstamo registrado; disponibles de Rayuela baja en 1	✓ Aprobada
19	Préstamos	Mismo libro dos veces al mismo usuario	Repetir la prueba 18	«Este usuario ya tiene ese libro prestado…»	✓ Aprobada
20	Préstamos	Libro sin ejemplares	Prestar `Rayuela` a otro usuario hasta agotarlo	«No hay ejemplares disponibles…»	✓ Aprobada
21	Préstamos	Fecha futura	Fecha posterior a hoy	El selector no la permite / mensaje de fecha	✓ Aprobada
22	Devoluciones	Registrar devolución	Seleccionar el préstamo de la prueba 18	Estado `Devuelto`, fecha guardada, disponibles vuelve a subir	✓ Aprobada
23	Devoluciones	Devolver un préstamo ya devuelto	Seleccionar uno `Devuelto`	El botón *Registrar devolución* está deshabilitado	✓ Aprobada
24	Préstamos	Eliminar préstamo activo	Seleccionar uno `Prestado`	Se elimina y el ejemplar vuelve al inventario	✓ Aprobada
25	Préstamos	Filtrar por estado	Estado = `Devuelto`	Solo préstamos devueltos	✓ Aprobada
26	Reportes	Generar los 10 reportes	Elegir cada uno	Cada uno muestra su tabla y el conteo de registros	✓ Aprobada

N.º	Módulo	Prueba	Datos de entrada	Resultado esperado	Resultado
27	Reportes	Exportar a CSV	Reporte *Préstamos activos*	Archivo `.csv` que abre en Excel con tildes correctas	✓ Aprobada

14. Conclusiones

- Se desarrolló una aplicación de escritorio funcional que cubre los módulos pedidos: libros, autores, editoriales, usuarios, préstamos, devoluciones y reportes, conectada a SQL Server.
- Separar el código en cuatro proyectos facilitó ubicar cada responsabilidad: ningún formulario contiene SQL y ninguna consulta decide qué mensaje ve el usuario.
- Dejar que la propia base de datos haga cumplir reglas (claves foráneas, `UNIQUE`, `CHECK` y el índice único filtrado) añade una segunda barrera de seguridad a las validaciones del código.
- Usar transacciones en préstamos y devoluciones evita que el inventario quede descuadrado si algo falla a mitad de la operación.
- Heredar de una plantilla común (`FormModulo`) redujo mucho el código repetido de los formularios y mantuvo una interfaz uniforme.
- Las consultas parametrizadas centralizadas en `Ejecutor` previenen la inyección SQL.
- El control de versiones con Git y GitHub dejó un historial de los cambios del proyecto y facilita su revisión.

15. Recomendaciones

- Agregar un **inicio de sesión con roles** (administrador y bibliotecario), guardando las contraseñas cifradas.
- Incluir **fecha límite de devolución y multas** por retraso, con alertas de préstamos vencidos.
- Permitir **varios libros por préstamo** (tabla de detalle) si la biblioteca lo requiere.
- Agregar **pruebas unitarias** para las clases de negocio y reportes en PDF.
- Hacer **copias de seguridad periódicas** de `BibliotecaDB` .
- Mover la cadena de conexión a un lugar seguro (variables de entorno o secretos de usuario) cuando se use fuera de un entorno académico.

16. Referencias bibliográficas

Microsoft. (2024). *Windows Forms documentation*. Microsoft Learn. https://learn.microsoft.com/dotnet/desktop/winforms/

Microsoft. (2024). *C# documentation*. Microsoft Learn. https://learn.microsoft.com/dotnet/csharp/

Microsoft. (2024). *Microsoft.Data.SqlClient namespace*. Microsoft Learn. https://learn.microsoft.com/dotnet/api/microsoft.data.sqlclient

Microsoft. (2024). *SQL Server technical documentation*. Microsoft Learn. https://learn.microsoft.com/sql/sql-server/

Microsoft. (2024). *Transactions and concurrency (ADO.NET)*. Microsoft Learn. https://learn.microsoft.com/dotnet/framework/data/adonet/transactions-and-concurrency

GitHub. (2024). *GitHub Docs*. https://docs.github.com/

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2